PROJECT WALKTHROUGH

Login System and Personal Organizer

A beginner-friendly guide to how the single-file C++ application works

ACCOUNT ACCESS	PERSONAL TOOLS	LOCAL FILES
Sign-up, sign-in, account changes, and recovery after three failed password attempts.	Tasks, notes, expenses, contacts, and recent account activity after sign-in.	Account records and feature data are stored in local files beside the program when it runs.

What this project is

This is a Windows console application written in C++. The original account workflow has been extended with small, file-based organizer features. The source stays in one file and uses familiar building blocks: classes, vectors, loops, input validation, and file streams.

At a glance	Current implementation
Language	C++ compiled with MinGW g++ using C++17 mode
Program entry	main() creates LoginSystem and opens the main menu
Authentication	Username and password; up to three password attempts before recovery
Organizer features	Tasks, notes, expenses, contacts, and recent activity
Storage	users.dat plus separate text files for organizer data

Important: this is a learning project. Its numeric password transformation is reversible and is not secure password hashing. Use test accounts and test data only.

1. How the application flows

The main menu handles sign-up and sign-in. A successful sign-in opens the account panel. Each menu choice is routed to a feature handler, which loads that user's records, performs the requested action, and saves the updated records.

Step	What happens
1. Start	main() creates LoginSystem and calls MainMenu().
2. Choose	The user selects Sign Up, Login, or Exit.
3. Authenticate	The login code finds the account and checks the entered password.
4. Recover if needed	After three failed password attempts, the application asks for the recovery code.
5. Use account panel	After successful access, the user chooses account details, account changes, or an organizer feature.
6. Save	Feature handlers update local files and append activity events when applicable.

Sign-up checks

- Username is 3 to 32 letters, digits, dots, dashes, or underscores.
- Username lookup ignores case, so a duplicate with different capitalization is still found.
- Password is 8 to 100 characters and requires uppercase, lowercase, digit, and special characters.
- Password entry uses getch() and displays asterisks instead of the typed characters.
- A non-empty recovery code of up to 100 characters is saved with the account.

Sign-in and recovery

The program looks up the username, then allows three password attempts. On success it opens the user panel. If all attempts fail, the user can enter the recovery code and either open the account or change the password. The account panel can also change the password, username, and recovery code after its modification check.

Console input

ProjectTools::AskNumber reads a complete line, parses one integer, rejects extra characters, and checks the allowed range. This keeps menu input simple and avoids leaving extra characters in the input stream.

2. Main classes and responsibilities

The file is long, but the work is split into small classes. Each handler owns the file operations and menu actions for one area.

Class	Responsibility
Password	Masked password entry, basic password rules, and comparison with the stored numeric representation.
User	Stores username, recovery code, creation date, and password data for one account.
FileHandler	Finds, creates, and updates account records in users.dat.
ProjectTools	Shared text cleanup, date formatting, lowercase conversion, numeric menu input, pause, and separators.
ActivityLog	Appends actions and returns recent activity for a username.
WorkspaceData	Updates owner usernames in organizer files when an account name changes.
TaskHandler / NoteHandler	Load, save, display, search, and change task or note records.
ExpenseHandler / ContactHandler	Manage expense and contact records and summaries.
Modify / UserPanel / LoginSystem	Control account updates, post-login navigation, and the top-level flow.

How saving works

The account handler writes binary records. It writes each string as a length followed by that many bytes. Organizer handlers use standard text streams. They load lines into vectors, select records by username, make the changes in memory, and rewrite the file. Records use the pipe character as a separator, so fields reject or clean pipe and newline characters.

File	Information stored
users.dat	Username, numeric password representation, recovery code, account creation date.
tasks.txt	Task ID, owner, title, description, due date, priority, status, creation date.
notes.txt	Note ID, owner, title, one-line content, creation date.
expenses.txt	Expense ID, owner, category, amount, description, date.
contacts.txt	Contact ID, owner, name, phone, email, group, creation date.
activity.log	Timestamp, username, and a short account or organizer action.

Run from the project directory so relative file names resolve to this folder. The data files are intentionally excluded from Git because they may contain personal information.

3. Organizer features

Every organizer menu is available from the user panel. The stored username on each record keeps one account's visible records separate from another account's records.

Feature	Available actions	Main fields
Tasks	Show, add, edit, change status, search, delete, and summarize.	Title, description, due date, priority, status.
Notes	Show, add, edit, search, and delete.	Title and one-line note content.
Expenses	Show, add, search, delete, and summarize by category.	Category, amount, description, date.
Contacts	Show, add, edit, search, and delete.	Name, phone, email, group.
Activity	Show recent actions for the current account.	Timestamp, username, action.

Tasks

Tasks have a generated integer ID, priority (Low, Normal, or High), status (Open, In Progress, or Done), and an optional due date. The summary reports counts and task status information.

Notes

Notes use a title and a single-line content field. They can be listed, searched, edited, and deleted. Keeping notes one line matches the simple pipe-separated storage format.

Expenses

An expense has a category such as Food, Travel, Bills, Shopping, Education, or Other. The summary totals spending and groups it by category.

Contacts

Contacts store a name, phone number, email address, and group. The user can list, search, edit, and remove contacts.

Activity history

Sign-in attempts, account changes, sign-out, and selected organizer changes are appended to activity.log. The account panel shows recent events for the signed-in user.

The records are plain local files, not a shared database. Two program instances writing at the same time can overwrite each other's changes.

4. Build, run, and check the project

The project currently uses one C++ source file and Windows console headers. A MinGW installation with a working g++ compiler is enough; CMake and external password libraries are not used.

Build and start in PowerShell

```
cd "C:\path\to\LOGIN SYSTEM"
g++ -std=c++17 LOGIN_SYSTEM.cpp -o LOGIN_SYSTEM.exe
.\LOGIN_SYSTEM.exe
```

Replace the example path with the actual project folder. Run the executable while the current directory is the project folder so users.dat and the organizer files are created beside the source.

Manual test checklist

- Sign up with a valid username and password; confirm a duplicate username is rejected even with different capitalization.
- Try a too-short username, a username with spaces, and a password missing each required character group.
- Sign in with the correct password, then try an incorrect password and verify that only three attempts are offered.
- After three incorrect passwords, try the wrong recovery code and then the correct one. Try both recovery choices.
- Change password, username, and recovery code. Confirm the renamed account still sees its old organizer records.
- For each organizer, test adding a record, listing it, searching, editing where supported, deleting, and restarting the app to check persistence.
- Enter letters, an out-of-range number, and a number followed by extra text at numeric menus.
- Run from the project folder with missing data files and confirm the application can create records as needed.

There is no automated test suite in the current project. These are manual checks to perform after a successful local build; they are not claimed as completed test results.

Common build issue

Compile the current source as one file. A command that lists src/main.cpp, src/password_helper.cpp, or an include directory belongs to a different multi-file layout and will not work for this project version. The current source does not require bcrypt.h or the C++ filesystem library.

5. Limitations and interview explanation

Explaining what the project does well and where it is limited is part of presenting it responsibly.

Security boundaries

- The password transform is a reversible arithmetic formula, not password hashing.
- The recovery code is saved with the account record and is not cryptographically protected.
- There is no password salt, encryption-at-rest, database, timed lockout, or protection against concurrent file writes.
- This project is for learning and demonstration with test data. Do not use real passwords or recovery codes.

Good next steps

- Replace the custom password representation with a maintained password-hashing library and a salt.
- Use a database or transactional storage if records need reliable updates or concurrent access.
- Add automated unit tests for parsing, validation, account flows, and organizer record operations.
- Separate source files when the author is comfortable explaining each module and its dependencies.

One-minute explanation

I built a single-file C++ console application with account sign-up and sign-in, account recovery, and personal organizer features. I used classes to group account data and keep file operations and menus organized. The organizer stores tasks, notes, expenses, and contacts in separate text files, while account records use a binary file with length-prefixed strings. The project helped me practice classes, vectors, validation, menus, and file streams. Its password transformation is not secure hashing, so I use test data only and would replace it before any real deployment.

Key terms to be ready to explain

Term	Simple explanation
Class	A type that groups data and functions that work on that data.
Vector	A resizable list used to hold records while a handler searches or updates them.
File stream	C++ input/output objects used to read and write local files.
Validation	Checks that user input meets length, character, range, or field-format rules.
CRUD	Create, read, update, and delete operations for organizer records.

Companion files

README.md explains installation, running, and data files. INTERVIEW_QUESTIONS.md contains beginner-level questions and example answers. Use this guide to understand the main flow before reviewing LOGIN_SYSTEM.cpp.